Split App

30.11.2025

Besevler Industries

Yusuf Eker - 23291233

Hüseyin Furkan Kızıloğlu - 23291069

Aykut Kocabıyık - 23291585

Emir Süha Baş - 22290115

Ders Sorumlusu: Dr. Öğr. Üyesi Rukiye SAVRAN KIZILTEPE

İçindekiler

1. Giriş 4

1.1. Amaç 4

1.2. Kapsam 4

1.3. Genel Bakış 5

1.3.1. Proje Paydaşları 5

1.3.2. Problem Durumu 5

1.3.3. Mevcut Sistemlerin Analizi 5

1.3.4. Çözüm Önerisi 6

1.3.5. Kullanılacak Teknolojiler 6

2. Yazılım Gereksinimleri 7

2.1. Fonksiyonel Gereksinimler 7

2.2. Fonksiyonel Olmayan Gereksinimler 8

3. Veritabanı Tasarımı 10

3.1. Kavramsal Tasarım 10

3.1.1. Varlık Tasarımı 10

3.1.2. Varlık-İlişki Diyagramı 11

3.1.3. Varlık Nitelik Açıklaması 12

3.2. Mantıksal Tasarım 17

3.2.1. Haritalama 17

3.2.2. Mantıksal İlişkiler 18

3.3. Fiziksel Tasarım 19

3.3.1. Normalizasyon 19

3.3.2. Fiziksel Veri Tabanı Tabloları 21

4. Veritabanı Sorguları 28

4.1. İç İçe Sorgu Örneği 28

4.2. Group By ve Having İçeren Sorgu Örneği 29

4.3. Join İşlemi İçeren Sorgu Örneği 30

5. Veritabanı Ekran Görüntüleri 31

6. Uygulama İçi Ekran Görüntüleri 33

7. Proje Yönetimi ve Görev Dağılımı 34

1. Giriş

Günlük hayatta birçok durumda ortak harcama yapılmaktadır. Aynı evde yaşayan bireyler, arkadaş gruplarıyla çıkılan tatiller, piknik ve kamp organizasyonları gibi ortak etkinlikler sırasında yapılan masraflar, çoğu zaman karmaşık ve hataya açık şekilde takip edilmektedir. Split App, bu gibi durumlarda yapılan harcamaların adil ve şeffaf bir biçimde yönetilmesini amaçlayan mobil tabanlı bir çözüm olarak geliştirilmiştir. Kullanıcılar harcamaları kolayca kaydeder, sistem borç-alacak ilişkisini otomatik olarak hesaplar.

1.1 Amaç

Split App'in temel amacı, ortak harcama yapılan tüm sosyal ortamlarda:

- Kimin ne kadar ödeme yaptığının açık şekilde kayıt altına alınmasını sağlamak,
- Borçların hızlı, doğru ve otomatik olarak hesaplanmasını gerçekleştirmek,
- Hesaplaşma sürecini kolaylaştırarak finansal anlaşmazlıkları önlemek,
- Kullanıcıların güvenli ve pratik bir çözümle ortak giderleri yönetebilmesini desteklemek olarak tanımlanmaktadır.

1.2 Kapsam

Split App aşağıdaki temel işlevleri kapsamaktadır:

- Ortak harcama ekleme (kategori, açıklama, tutar, ödeyen kişi)
- Farklı gruplar oluşturma (ev, etkinlik, seyahat vb.)
- Otomatik borç dengeleme algoritması
- Harcama geçmişi takibi
- Kişi bazlı istatistiksel görünüm (kim ne kadar harcadı?)
- Grup hesap kapatma ve sıfırlama mekanizması
- Mobil ortamda erişilebilirlik ve kullanım kolaylığı

Bu kapsam sayesinde uygulama, farklı sosyal grupların finansal ihtiyaçlarına uyarlanabilir bir yapıda olacaktır.

1.3 Genel Bakış

1.3.1 Proje Paydaşları

- **Kullanıcılar:** Ortak gider yapılan grup üyeleri
- **Grup Yöneticileri:** Grup oluşturma ve düzenleme yetkisine sahip kullanıcılar
- **Geliştirme Ekibi:** Analiz, tasarım, yazılım geliştirme ve test süreçlerinden sorumlu kişiler
- **Sistem Yöneticileri:** Sunucu yönetimi, güvenlik ve veri sürekliliğini sağlayan ekip

1.3.2 Problem Durumu

Ortak giderler genellikle aşağıdaki problemleri ortaya çıkarır:

- Harcama takibi unutulabilir veya kaybolabilir
- Kimin ne kadar ödeme yaptığı hatırlanamaz
- Adil olmayan hesaplama şüphe ve güvensizlik yaratır
- Süreç çoğu zaman ertelemelere ve anlaşmazlıklara dönüşür
- Basit matematik hataları ilişkileri gereksiz gerer

Örneğin bir piknikte:

Biri mangalı, biri içecekleri, biri meyveyi almıştır. Eve dönüldüğünde “Kim ne verdi? Ne kadar düz hesap yapalım?” tartışması kaçınılmazdır. Bu gerçekler, dijital ve objektif bir çözüme olan ihtiyacı açıkça göstermektedir.

1.3.3 Mevcut Sistemlerin Analizi

Hali hazırda kullanılan yöntemler:

- Sözlü tahminler: hatalı
- Kâğıt defter/fişler: kaybolabilir
- Mesaj grupları: karmaşık, takip zor

- Bazı yabancı uygulamalar:
    - Türkçe destek zayıf
    - Grup kullanımında öğrenme eğrisi yüksek
    - Türkiye'de yaygın olmayan ödeme senaryoları

Dolayısıyla Split App daha erişilebilir, yerel ve odaklı bir çözüm sunacaktır.

1.3.4 Çözüm Önerisi

Split App ile:

1. Harcamalar tek tek kaydedilir
2. Kişi sayısına göre eşit/ayarlanabilir bölüşüm yapılır
3. Borçlar minimum işlemle dengelenir
4. Son durumda herkesin durumu grafik ve listelerle gösterilir

Adalet sayılarla kanıtlanmış olur. Kimse "ben fazla verdim" diye içten içe söylenmez.

1.3.5 Kullanılacak Teknolojiler

- **Mobil Geliştirme:** React Native (Expo)
- **Veri tabanı & Doğrudan Erişim:** Supabase (**PostgreSQL)**
- **Kimlik Doğrulama: Supabase Auth (JWT)**
- **Backend (İş Kuralları / API): FastAPI (Python)**
- **Depolama: Supabase Storage (fiş/foto vs.)**
- **Bildirimler: Expo Notifications (gerekirse FastAPI webhook ile tetik)**
- **Versiyon Yönetimi:** GitHub

Seçilen teknolojiler, ölçeklenebilir ve sürdürülebilir bir uygulama geliştirilmesine olanak tanır.

2. Yazılım Gereksinimleri

2.1. Fonksiyonel Gereksinimler

Bu alt bölümde Split App'in fonksiyonel gereksinimlerine yer verilecektir. Bu bölümdeki tabloda yer alan gereksinim kimliği bölümü benzersiz gereksinim numarasını tanımlar. Öncelik numarası gereksinimin önceliğini belirtir. Daha yüksek öncelik numarası gereksinimin daha öncelikli olduğu anlamına gelir. Öncelik numarası birden küçük beşten büyük olamaz. Açıklama kısmında ise gereksinimin kısa açıklaması bulunur.

Gereksinim Kimliği	Öncelik Numarası	Açıklama
REQ.F.01	5	Kullanıcı kaydolabilmelidir (e-posta veya telefon).
REQ.F.02	5	Kullanıcı giriş yapabilmelidir (şifre doğrulama).
REQ.F.03	3	Şifre sıfırlama işlemi yapılabilmelidir.
REQ.F.04	3	Kullanıcı profil bilgilerini düzenleyebilmelidir.
REQ.F.05	5	Kullanıcı yeni bir grup oluşturabilmelidir.
REQ.F.06	5	Kullanıcı gruba başka üyeler ekleyebilmelidir.
REQ.F.07	4	Grup yöneticisi üyeleri çıkarabilmelidir.
REQ.F.08	5	Kullanıcı birden fazla gruba katılabilmelidir.
REQ.F.09	5	Kullanıcı harcama ekleyebilmelidir (tutar, açıklama, kategori).
REQ.F.10	5	Harcamayı kimin ödediği bilgisi seçilebilmelidir.
REQ.F.11	5	Harcama tüm grup üyelerine otomatik dağıtılmalıdır.
REQ.F.12	5	Harcamalar düzenlenebilir veya silinebilir olmalıdır.

REQ.F.13	5	Kullanıcı borç ödeme işlemini işaretleyebilmelidir.
REQ.F.14	4	Sistem kimin kime ne kadar borçlu olduğunu otomatik hesaplamalıdır.
REQ.F.15	3	Dengeleme işlemi sonrası sonuçlar güncellenmelidir.

Tablo 1: Fonksiyonel Gereksinimler

2.2. Fonksiyonel Olmayan Gereksinimler

Bu alt bölümde Split App uygulamasının, sistemin performansı, güvenliği, erişilebilirliği gibi teknik özelliklerine yönelik belirlenen fonksiyonel olmayan gereksinimleri detaylı bir şekilde açıklanmaktadır.

Gereksinim Kimliği	Öncelik Numarası	Açıklama
REQ.NF.01	2	Hesaplama işlemleri en fazla **1 saniye** içinde tamamlanmalıdır.
REQ.NF.02	3	Arayüz sade, öğrenmesi kolay olmalıdır.
REQ.NF.03	4	Her kullanıcı yalnızca kendi gruplarını ve verilerini görebilmelidir.
REQ.NF.04	4	Android ve iOS platformlarında sorunsuz çalışmalıdır.
REQ.NF.05	4	Sistem bilgisayar ve tablet cihazlarla uyumlu olmalıdır.
REQ.NF.06	3	Veri tabanı günde en az bir kez yedeklenmelidir.
REQ.NF.07	3	Sistem eş zamanlı 100 kullanıcıyı desteklemelidir.

REQ.NF.08	2	Yeni özellikler eklendiğinde mevcut yapıyı bozmamalıdır.
REQ.NF.09	3	Sistem açık kaynaklı teknolojilerle geliştirilecektir.
REQ.NF.10	4	Uygulama %95 ve üzeri kullanılabilirlik sağlamalıdır.
REQ.NF.11	5	Kullanıcıya kişisel veri işleme hakkında bilgi verilmelidir.
REQ.NF.12	5	Kullanıcı şifreleri hash yöntemi ile saklanmalıdır.
REQ.NF.13	3	Hatalar loglanmalı ve geliştiriciye bildirilmelidir.
REQ.NF.14	4	Yönetici paneline sadece yetkili kullanıcılar erişebilmelidir.
REQ.NF.15	4	Sistem her 10 saniyede bir bağlantı durumu kontrol etmelidir.
REQ.NF.16	5	Uygulama yeni tarayıcı güncellemeleriyle uyumlu olmalıdır.

Tablo 2: Fonksiyonel Olmayan Gereksinimler

3. Veri Tabanı Tasarımı

3.1. Kavramsal Tasarım

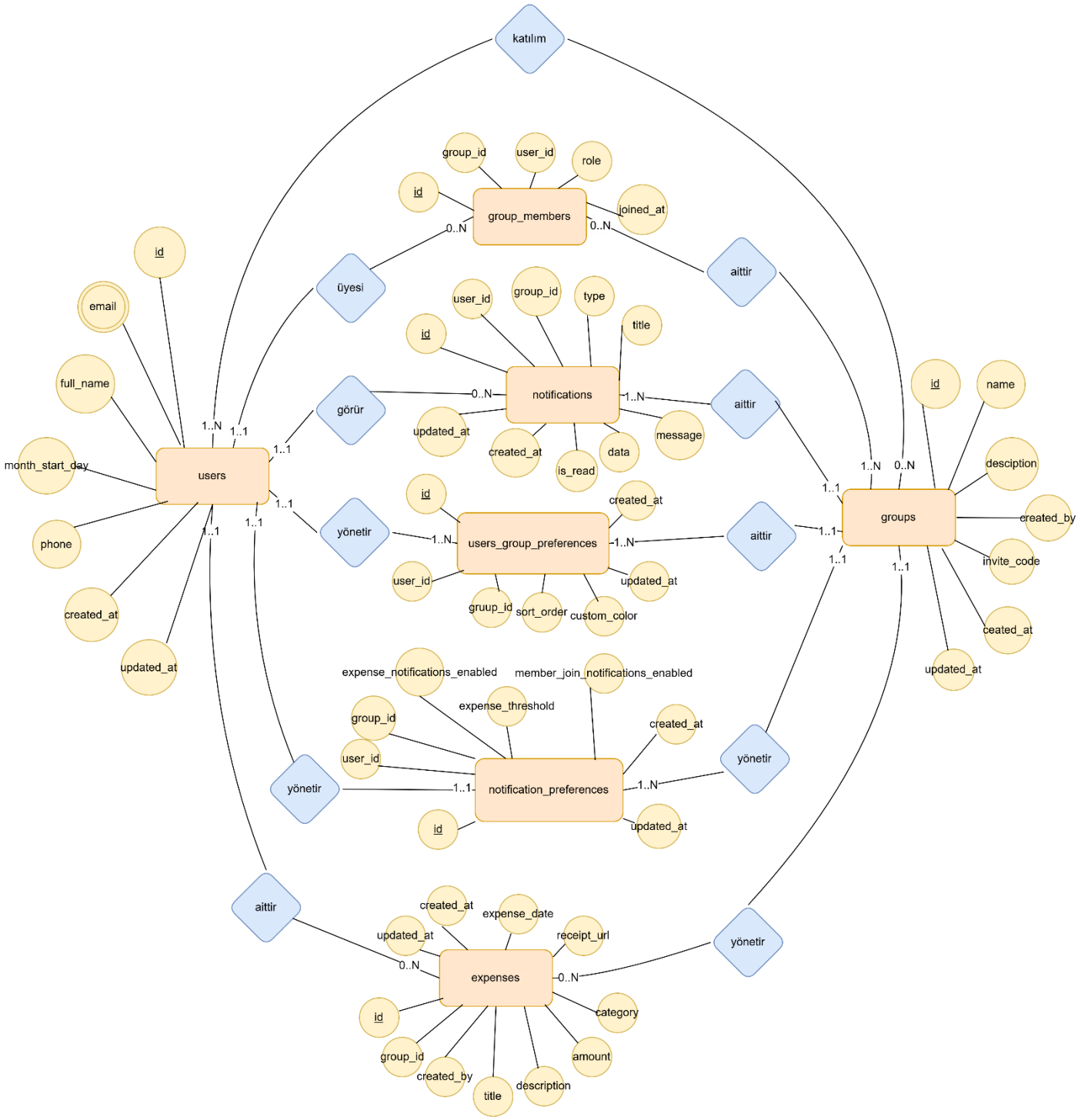
3.1.1. Varlık Tanımı

Varlık	Açıklama
users	Uygulamaya kaydolup giriş yapan, gruplara katılan veya yeni grup oluşturan tüm kişileri temsil eder. Her kullanıcı kendi harcamalarını ekleyebilir, gruplarda borç–alacak ilişkilerine dâhil olur ve profil ayarlarını yönetebilir.
group	Ev, etkinlik, tatil, piknik gibi ortak harcama ortamlarını temsil eden yapılardır. Her grup, içindeki üyeler ve o üyelerin eklediği harcamalardan oluşur.
expenses	Bir kullanıcı tarafından eklenen; tutar, kategori, açıklama ve "ödeyen kişi" bilgilerini tutan kayıtları ifade eder. Borçların nasıl bölüşüleceği bu tabloda yer alan verilere göre hesaplanır.
notification_preferences	Kullanıcının hangi bildirimleri almak istediğini belirleyen varlıktır.
user_group_preferences	Her kullanıcının her grup için ayrı ayrı tuttuğu bildirim tercihleri burada tutulur.
notifications	Sistemin ürettiği tüm bildirim kayıtları burada saklanır.
group_members	Hangi kullanıcının hangi gruba bağlı olduğunu gösterir. Ayrıca rol bilgisi (grup yöneticisi/üye) bu varlıkta tutulur.

Tablo 3: Varlık Türleri Tablosu

3.1.2. Varlık-İlişki Diyagramı

Bu bölümde Şekil 1'de SplitApp uygulamasına ait ER diyagramına yer verilmiştir.



Şekil 2: SplitApp Veri Tabanı ER Diyagramı

3.1.3. Varlık Nitelikleri Açıklaması

Bu bölümde Şekil 2'de verilen ER diyagramında bulunan varlıkların nitelikleri anlatılacaktır.

3.1.3.1. Users Varlığı

users	
Nitelik	**Açıklama**
id	Kullanıcıya özel bir ID tutar; sistemdeki birincil anahtardır.
email	Kullanıcının sisteme giriş için kullandığı e-posta adresini tutar.
full_name	Kullanıcının tam adını tutar.
phone	Kullanıcının telefon numarasını tutar (opsiyoneldir).
created_at	Kullanıcı hesabının oluşturulma tarihini tutar
updated_at	Kullanıcı hesabının son güncellenme zamanını tutar.

month_start_day	Kullanıcının aylık hesaplamalarının başlangıç gününü tutar.

Tablo 4: Users Varlığı Nitelikleri

3.1.3.2. Groups Varlığı

groups	
Nitelik	**Açıklama**
id	Her grup için benzersiz ID tutar.
name	Grubun adını tutar.
desciption	Gruba ait açıklama bilgisini tutar.
created_by	Grubu oluşturan kullanıcıyı belirler.
invited_code	Gruba özel davet kodunu tutar; otomatik olarak üretilir.
created_at	Grubun oluşturulma zamanını tutar.
updated_at	Grubun son güncellenme zamanını tutar.

Tablo 5: Grups Varlığı Nitelikleri

3.1.3.3. Expenses Varlığı

expenses	
Nitelik	**Açıklama**
id	Harcama kaydı için benzersiz ID tutar.
group_id	Harcamanın ait olduğu grubu belirtir (FK → groups.id).
created_by	Harcamayı ekleyen kullanıcıyı belirtir (FK → users.id).
title	Harcamanın kısa açıklama başlığını tutar.
description	Harcamaya ilişkin açıklama bilgisini tutar.
amount	Harcamanın parasal değerini tutar.
category	Harcamanın kategorisini tutar (varsayılan: “other”).

receipt_url	Harcama fişinin URL bilgisini tutar (opsiyoneldir).
expense_date	Harcamanın gerçekleştiği tarihi tutar.
created_at	Harcamanın sisteme eklenme zamanını tutar.
updated_at	Harcama kaydının güncellenme zamanını tutar.

Tablo 6: Expenses Varlığı Nitelikleri

3.1.3.4. Notification_preferences Varlığı

notification_preferences	
Nitelik	**Açıklama**
id	Bildirim tercihi kaydı için benzersiz ID tutar.
users_id	Tercihi yapan kullanıcıyı belirtir (FK → users.id).
group_id	Tercihin ait olduğu grubu belirtir (FK → groups.id).
expense_notifications_enabled	Harcama bildirimlerinin aktif olup olmadığını tutar (true/false).
expense_threshold	Harcamalar için minimum bildirim eşiğini tutar.
member_join_notifications_enabled	Üyelik bildirimlerinin aktif olup olmadığını tutar.
created_at	Tercih kaydının oluşturulduğu zamanı tutar.
updated_at	Tercih kaydının güncellendiği zamanı tutar.

Tablo 7: Notification_preferences Varlığı Nitelikleri

3.1.3.5. User_group_preferences Varlığı

user_group_preferences	
Nitelik	**Açıklama**
id	Ayar kaydı için benzersiz ID tutar.
user_id	Ayar kaydı için benzersiz ID tutar.
group_id	Ayarın ait olduğu grubu belirtir (FK → groups.id).
sort_order	Bu grubun kullanıcıya özel sıralama indeksini tutar.
custom_color	Kullanıcının grup için seçtiği özel rengi tutar.
updated_at	Ayar kaydının güncellenme zamanı.
created_at	Ayar kaydının oluşturulma zamanı.

Tablo 8: User_group_preferences Varlığı Nitelikleri

3.1.3.6. Notifications Varlığı

notifications	
Nitelik	**Açıklama**
id	Bildirim kaydı için benzersiz ID tutar.
user_id	Bildirimin gönderildiği kullanıcıyı belirtir (FK → users.id).
group_id	Bildirimin ait olduğu grubu belirtir; opsiyoneldir (FK → groups.id).
type	Bildirim türünü tutar (expense_added, member_joined, group_created, system).
title	Bildirimin başlık bilgisi.
message	Bildirimin açıklama/metin içeriği.
data	Bildirime özel ek JSON verisini tutar.
is_read	Bildirimin okunup okunmadığını belirtir (true/false).
Created_at	Bildirimin oluşturulma zamanını saklar.
Updated_at	Bildirimin en son düzenlenme zamanını gösterir.

Tablo 9: Notifications Varlığı Nitelikleri

3.1.3.7. Group_members Varlığı

group_members	
Nitelik	**Açıklama**
id	Üyelik kaydı için benzersiz ID tutar.
group_id	Üyenin ait olduğu grubu belirtir (FK → groups.id).
user_id	Gruba üye olan kullanıcıyı belirtir (FK → users.id).
role	Kullanıcının gruptaki rolünü tutar (admin/member).
joined_at	Kullanıcının gruba katılma zamanını tutar.

Tablo 10: Group_members Varlığı Nitelikleri

3.2. Mantıksal Tasarım

3.2.1. Haritalama (Mapping)

Bu bölümde SplitApp sisteminin veri tabanı için haritalama (mapping) yapılacaktır.

- Users(**PK id**, email, full_name, phone, created_at, updated_at, month_start_day)

- Groups(**PK id**, name, description, **FK** created_by **REFERENCES Users**(id), invite_code, created_at, updated_at)

- GroupMembers(**PK id**, **FK** group_id **REFERENCES Groups**(id), **FK** user_id **REFERENCES** Users**(**id**)**, role, joined_at)

- Expenses(**PK** id, **FK** group_id **REFERENCES Groups(**id**)**, **FK** created_by **REFERENCES Users(**id**)**, title, description, amount, category, receipt_url, expense_date, created_at, updated_at)

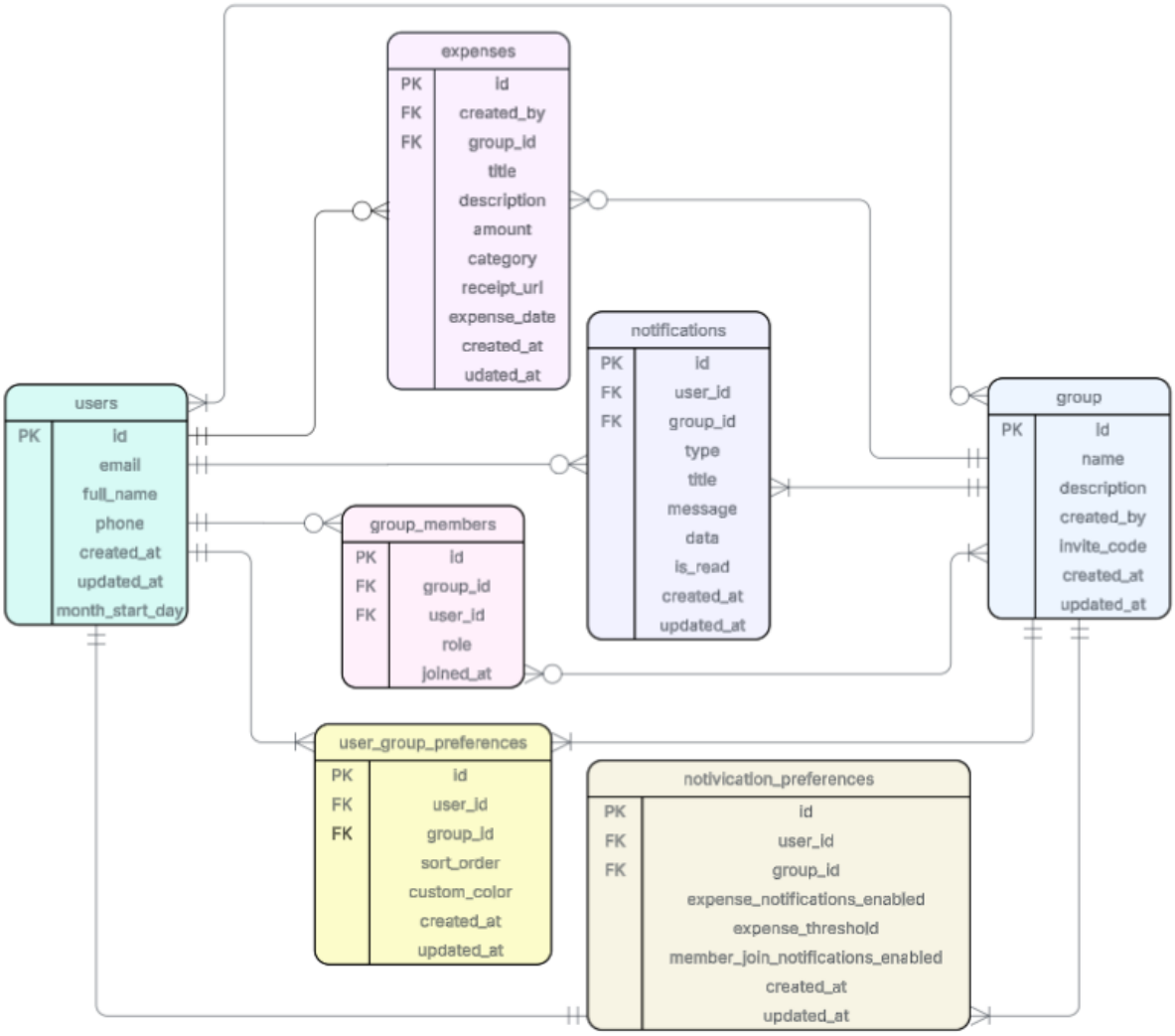
- NotificationPreferences(**PK** id, **FK** user_id **REFERENCES Users**(id), **FK** group_id **REFERENCES Groups**(id), expense_notifications_enabled, expense_threshold, member_join_notifications_enabled, created_at, updated_at)

- Notifications(**PK** id, **FK** user_id **REFERENCES Users**(id), **FK** group_id **REFERENCES Groups**(id), type, title, message, data, is_read, created_at, updated_at)

- UserGroupPreferences(**PK** id, **FK** user_id **REFERENCES Users**(id), **FK** group_id **REFERENCES Groups**(id), sort_order, custom_color, created_at, updated_at)

3.2.2. Mantıksal İlişkiler

Bu bölümde Şekil 2'de SplitApp uygulamasına ait Mantıksal İlişkiler diyagramına yer verilmiştir.



Şekil 2: SplitApp Veri Tabanı Mantıksal İlişkiler Diyagramı

3.3. Fiziksel Tasarım

3.3.1. Normalizasyon

Bu kısımda veri tabanı bileşenlerinin 1NF (1. Normal Form), 2NF (2. Normal Form) ve 3NF (3. Normal Form) uyumluluğu test edilecek ve uymayanlar gerekli forma uyarlanacaktır.

3.3.1.1. 1.Normal Form

Veri tabanı tablolarının 1. normal forma uyması için gereken düzenlemeler şu şekildedir:

- Aynı niteliğe sahip tüm değerler tutarlı bir biçimde aynı veri tipinde tanımlanmalıdır.
- Tablodaki nitelik alanları benzersiz, anlamlı ve ayırt edici olacak şekilde düzenlenmelidir.
- Tüm nitelik değerleri atomik bir yapıda olacak biçimde organize edilmelidir.

SplitApp üzerinde kavramsal modelden mantıksal modele taşınan users, groups, group_members, expenses, notifications, notification_preferences ve user_group_preferences tablolarının ilk halleri incelendiğinde tüm alanların atomik ve tekrarsız değerler içerdiği görülmüştür. Bu tablolarda bir hücre içinde birden fazla bilgi bulundurma, listeleme ya da çoklu veri yapıları yer almamaktadır. Bu nedenle mapping üzerinden çıkarılan tabloların ilk halleri 1. Normal Form'a uygundur.

3.3.1.2. 2. Normal Form

Veri tabanının 2. normal forma uyması için gerekenler aşağıdadır:

- Tüm tabloların 1. Normal Form'a uyuyor olması gerekir.
- Veriler arasında kısmi fonksiyonel bağımlılık olmamalıdır.

SplitApp veri tabanı incelendiğinde tabloların hiçbirinde bileşik birincil anahtar kullanılmadığı tespit edilmiştir. Tüm tablolar tek birincil anahtar olan id üzerinden tanımlanmıştır. Bu nedenle herhangi bir niteliğin birincil anahtarın yalnızca bir bölümüne bağımlı olması gibi bir durum ortaya çıkmamıştır.

Bu kapsamda özellikle group_members, notification_preferences ve user_group_preferences tabloları, teorik olarak bileşik anahtar ile tanımlanabilir aday yapılar olmasına rağmen tasarımda tekil birincil anahtar kullanılmıştır. Böylece kısmi bağımlılık ihtimali tamamen ortadan kaldırılmıştır.

Bu yapılan tasarım tercihleri sonucunda tabloların tamamı 2. Normal Form'a uygun hale getirilmiştir.

3.3.1.3. 3. Normal Form

Veri tabanının 3. normal forma uyması için gerekenler aşağıdadır:

- Tüm tabloların 2. Normal Form'a uyuyor olması gerekir.
- Tablodaki nitelikler arasında geçişli fonksiyonel bağımlılık olmamalıdır.

Bu koşullar doğrultusunda SplitApp veri tabanı tabloları karşılaştırıldığında nitelikler arasında herhangi bir geçişli bağımlılık bulunmadığı görülmüştür. Tablolarda yer alan

nitelikler yalnızca kendi birincil anahtarları tarafından belirlenmekte, bir nitelik başka bir nitelik üzerinden dolaylı bir bağımlılık ilişkisi göstermemektedir.

Örneğin harcamalara ilişkin bilgiler yalnızca expenses tablosundaki id alanına bağlıdır; kullanıcı bilgileri yalnızca users tablosundaki id alanına bağlıdır. Benzer şekilde bildirimlere ilişkin bilgiler de yalnızca notifications tablosundaki birincil anahtar ile ilişkilendirilmiştir. Bu nedenle tablolar arasında {anahtar olmayan nitelik} → {başka anahtar olmayan nitelik} biçiminde bir bağımlılık ilişkisi tespit edilmemiştir.

Bu yapılan incelemeler sonucunda SplitApp veri tabanı tabloları 3. Normal Form'a uygun hale getirilmiştir.

Bütün bu normalizasyon aşamaları sonucunda oluşan tablolar aşağıda gösterildiği gibidir.

expenses

id	group_id	created_by	title	descriptio	amoun	categor	receipt_u	expense_dat	created_at	updated_at
01e3e125-45	922310f7-56bf-4e5c-	6e74ecd4-992a-4	Market	a101	100.00	rent		30.08.2025	2025-08-30 23:4	2025-08-30 23:44:07.060028+00
22961f31-0a	922310f7-56bf-4e5c-	6e74ecd4-992a-4	Elektrik FaturasÃ±		100.00	other		9.09.2025	2025-09-09 19:4	2025-09-09 19:43:30.910895+00
2a16fe53-45	922310f7-56bf-4e5c-	6e74ecd4-992a-4	ww		1.00	other		31.08.2025	2025-08-31 02:1	2025-08-31 02:13:08.311976+00
5461dd62-8	922310f7-56bf-4e5c-	6e74ecd4-992a-4	yeni harcama		85.00	other		9.09.2025	2025-09-09 19:4	2025-09-09 19:47:21.870569+00
84d4d344-9	922310f7-56bf-4e5c-	6e74ecd4-992a-4	Elektrik fatura	faturayÃ± yine	440.00	bills		30.08.2025	2025-08-30 23:4	2025-08-30 23:48:29.130114+00

groups

id	name	description	created_by	invite_code	created_at	updated_at
1849e7af-4b	Ofisim		6e74ecd4-992a	B2ABDC0D	2025-09-04 20	2025-09-04 20:57:22.933531+00
372d9ecd-9c	Heyy		6e74ecd4-992a	DDDC09DE	2025-09-04 20	2025-09-04 20:57:45.220105+00
922310f7-56	Genel	Furkan ve Yusuf'un Evi	6e74ecd4-992a	8452B0AF	2025-08-30 13	2025-11-20 17:49:11.732104+00
bb29fb9d-6c	Evim		262524e9-a711	6083733C	2025-08-31 00	2025-08-31 00:00:32.619775+00
d0556727-35	hh		6e74ecd4-992a	613AA96B	2025-09-04 21	2025-09-04 21:53:54.178182+00

notification_preferences

id	user_id	group_id	expense_notifications_enabled	expense_threshold	member_join_notifications_enabled	created_at	updated_at
409ebdc0-	6e74ecd4-992a-	922310f7-56bf-4e	true	0.00	true	2025-09-02 20:05:17.893989+00	2025-11-20 17:43:27.93596+00
74c2d9cc-	6e74ecd4-992a-	1849e7af-4b3e-45	true	0.00	true	2025-09-04 20:57:23.438669+00	2025-11-20 17:43:28.470612+00
89d3ec80-	6e74ecd4-992a-	d0556727-3586-45	true	0.00	true	2025-09-04 21:53:54.64406+00	2025-11-20 17:43:29.007551+00
e581242c-	6e74ecd4-992a-	372d9ecd-9dcd-4	true	0.00	true	2025-09-04 20:57:45.682271+00	2025-11-20 17:43:28.755138+00

user_group_preferences

id	user_id	group_id	sort_order	custom_color	updated_at	created_at
16331e9c-3a3d-	6e74ecd4-992a-4	d0556727-3586-45b5-8b	5	#FF6B6B	2025-09-08 02:25:17.049943+00	2025-09-04 21:53:55.142099+00
470347a0-e5df-	262524e9-a711-4	922310f7-56bf-4e5c-a9f	1		2025-09-01 01:20:13.158827+00	2025-09-01 01:19:07.324571+00
493bacab-78c0	262524e9-a711-4	bb29fb9d-6cd6-433b-99	3		2025-09-01 01:20:13.606934+00	2025-09-01 01:19:06.534035+00
51197082-2e9b-	6e74ecd4-992a-4	372d9ecd-9dcd-40bf-a5	3	#00FF94	2025-09-08 02:25:16.592113+00	2025-09-04 20:57:48.969893+00
780e0b3c-3734-	6e74ecd4-992a-4	1849e7af-4b3e-452b-b1	2	#00D4FF	2025-09-08 02:25:16.367039+00	2025-09-04 20:57:27.480762+00
f04be97b-9419-	6e74ecd4-992a-4	922310f7-56bf-4e5c-a9f	1	#FF0080	2025-09-08 02:25:16.121324+00	2025-08-31 16:20:17.037276+00

users

id	email	full_name	phone	created_at	updated_at	month_start_day
262524e9-	huseyinfurkankiziloglu@gmail.com	Ahmet Yılmaz		2025-08-30 23:59:59.628339+00	2025-09-01 01:19:37.053553+00	15
6e74ecd4-	huseyinfurkankiziloglu1@gmail.co	Furkan Kızıloğlu	5517305830	2025-08-30 12:56:44.141444+00	2025-09-09 08:34:56.111359+00	15
7eb06a3d-	furkan@gmail.com	furkan Baba		2025-08-30 12:58:54.023281+00	2025-08-31 00:04:44.527451+00	1
9b98ed9c-	cc@gmail.com	Ceyhun Cenk		2025-08-31 11:15:30.554358+00	2025-08-31 11:15:30.554358+00	1
fde0133b-	abc@mail.com	Furkan Kızıl		2025-11-20 07:25:52.179122+00	2025-11-20 07:25:52.179122+00	1

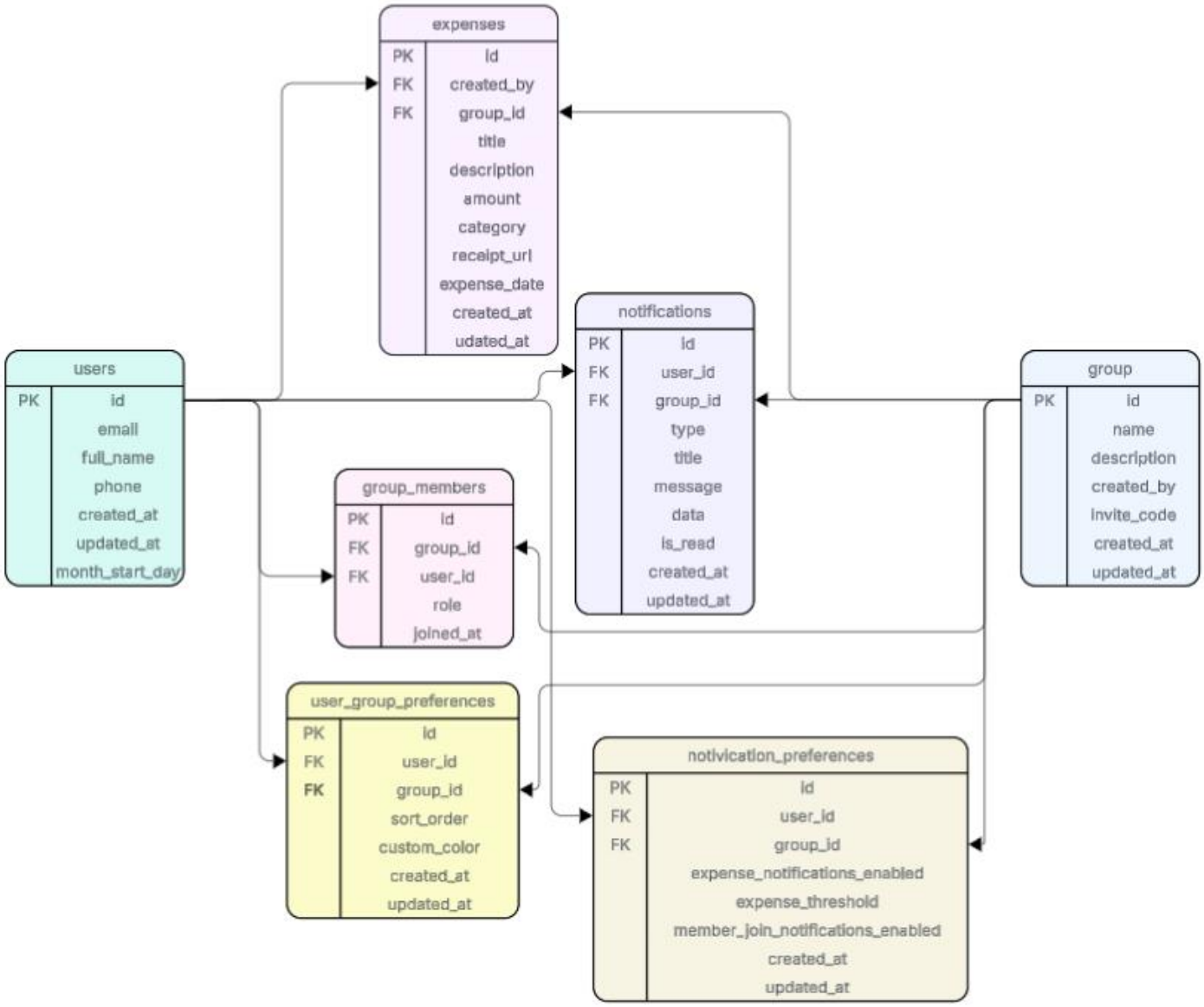
group_members

id	group_id	user_id	role	joined_at
0f555f9d-8634-4b7c-a	922310f7-56bf-4e5c-a9f	9b98ed9c-8de3-4b63-9	member	2025-08-31 11:16:10.454785+00
1507537f-172e-48ec-9	bb29fb9d-6cd6-433b-99	262524e9-a711-49d5-9	admin	2025-08-31 00:00:32.916609+00
1b2932c9-f9ea-402a-9	922310f7-56bf-4e5c-a9f	262524e9-a711-49d5-9	member	2025-08-31 01:35:29.817018+00
1d852c0b-4f5f-43db-8	372d9ecd-9dcd-40bf-a5	6e74ecd4-992a-4fe7-86	admin	2025-09-04 20:57:45.464237+00
33877fc9-6cb1-4e81-9	d0556727-3586-45b5-8b	6e74ecd4-992a-4fe7-86	admin	2025-09-04 21:53:54.425452+00
8f1a2de0-632f-4b1c-b	1849e7af-4b3e-452b-b1	6e74ecd4-992a-4fe7-86	admin	2025-09-04 20:57:23.175968+00
ba2ce303-26d6-429f-a	922310f7-56bf-4e5c-a9f	6e74ecd4-992a-4fe7-86	admin	2025-08-30 13:52:44.737112+00

Şekil 3: Normalizasyon Sonrası SplitApp Tabloları

3.3.2. Fiziksel Veri Tabanı Tabloları

Bu bölümde projenin fiziksel tasarımı içerisinde fiziksel veri tabanı tabloları anlatılacaktır. Şekil 4'te tablolarda primary key ve foreign key ilişkileri görülebilir.



Şekil 4: Fiziksel Veri Tabanı Tabloları

3.3.2.1. Users Tablosu

- **id (PK):** Kullanıcıların sistemde eşsiz olarak tanımlanabilmesi için primary key olarak belirlenmiştir. SplitApp mimarisinde kullanıcı kimlikleri farklı tablolarda yer alan veri ilişkilerinin temelini oluşturduğu için benzersiz bir anahtar gereklidir. Evrensel benzersiz kimlik sağladığı için UUID veri tipi kullanılmıştır.

- **email:** Kullanıcıların sisteme giriş yapabilmesini sağlayan iletişim bilgisidir. E-posta adresi metinsel bir değer olduğu için varchar veri tipi tanımlanmıştır.

- **full_name:** Kullanıcının tam adını saklamak için kullanılır. Bu alan metin içerdiğinden varchar veri tipi tercih edilmiştir.

- **phone:** Kullanıcıların isteğe bağlı olarak telefon bilgilerini depolamak için kullanılır. Numara formatındaki bir metin olduğu için varchar tipi ile tanımlanmıştır.

- **created_at:** Kullanıcının sisteme kayıt olduğu zamanı belirtmek için kullanılır. Tarih–saat bilgisini hassas şekilde kaydetmek amacıyla timestamp veri tipi kullanılmıştır.

- **updated_at:** Kullanıcı bilgilerinin en son ne zaman güncellendiğini belirtir. Bu alan da timestamp veri tipi ile tanımlanmıştır.

- **month_start_day:** Kullanıcının hesabını açma tarihi tutulur.

3.3.2.2. Groups Tablosu

- **id (PK):** Grupların eşsiz olarak ayırt edilmesini sağlamak için primary key olarak tanımlanmıştır. Grup kimliklerinin farklı ilişkilerde (harcamalar, kullanıcı tercihleri, bildirimler) referans alınması gerektiğinden UUID veri tipi seçilmiştir.

- **name:** Grubun adını belirtmek için kullanılır. Metinsel yapıda olduğu için varchar kullanılmıştır.

- **description:** Gruba ilişkin açıklama bilgisini tutar. Uzun metinlerin saklanabilmesi için text veri tipi tercih edilmiştir.

- **created_by (FK):** Grubu oluşturan kullanıcıyı belirtmek için foreign key olarak tanımlanmıştır. Users tablosundaki id alanına referans eder. Veri tipi UUID'dir.

- **invite_code:** Gruplara katılımın sağlanabilmesi için oluşturulan benzersiz davet kodudur. Metin yapısında olması nedeniyle varchar veri tipi kullanılmıştır.

- **created_at:** Grubun ne zaman oluşturulduğunu göstermek için timestamp veri tipi ile tanımlanmıştır.

- **updated_at:** Grup bilgilerinin en son güncellendiği zamanı belirtir. Aynı şekilde timestamp veri tipi kullanılmıştır.

3.3.2.3. GroupMembers Tablosu

- **id (PK):** Grup–kullanıcı ilişkisinin tekilleştirilmesi amacıyla primary key olarak belirlenmiştir. Bu alan, hem kullanıcı hem de grup verilerinin tek bir satırda birleşmesini sağladığı için UUID veri tipi ile tanımlanmıştır.

- **group_id (FK):** Kullanıcının dahil olduğu grubu belirtir. Groups tablosundaki id alanına referans eder. Veri tipi UUID'dir.

- **user_id (FK):** Grup üyesinin kimliğini belirtir. Users tablosundaki id alanına referans eder. UUID veri tipi kullanılmıştır.

- **role:** Kullanıcının gruptaki rolünü belirtir (admin, member). Metin tabanlı sınırlı değer içereceği için varchar(10) olarak tanımlanmıştır.

- **joined_at**: Kullanıcının gruba ne zaman katıldığını göstermek için timestamp veri tipi tanımlanmıştır.

3.3.2.4. Expenses Tablosu

- **id (PK):** Harcamaların sistem içinde benzersiz bir şekilde tanımlanabilmesi için primary key olarak belirlenmiştir. Her harcama kaydının başka tablolarda (ör. notifications) referans edilebilmesi için tekil bir kimlik gereklidir. Evrensel benzersiz tanımlayıcı sağladığı için UUID veri tipi kullanılmıştır.

- **group_id (FK):** Harcamaların ait olduğu grubu belirtmek için foreign key olarak tanımlanmıştır. Groups tablosundaki id alanına referans eder. Bu ilişki sayesinde harcamalar doğru kullanıcı grubuyla ilişkilendirilir. Veri tipi UUID'dir.

- **created_by (FK):** Harcamayı kimin eklediğini belirtir. Users tablosundaki id alanına referans eden foreign key olarak tanımlanmıştır. Veri tipi UUID olarak seçilmiştir.

- **title:** Harcamaya ilişkin kısa başlık bilgisini tutar. Metinsel bir içerik olduğu için varchar veri tipi uygun görülmüştür.

- **description:** Harcamaya dair açıklama bilgisini içerir. Uzun metinlerin saklanabilmesi için text veri tipi kullanılmıştır.

- **amount:** Harcamada yer alan tutarı temsil eder. Sayısal bir değer olduğu için numeric veri tipi seçilmiştir.

- **category:** Harcamaların sınıflandırılması amacıyla kategoriyi belirtir. Metin içereceği için varchar(30) veri tipi seçilmiştir. Maksimum uzunluk sınırlaması kategorilerin standartlaştırılmasını sağlar.

- **receipt_url:** Harcamaya ait fiş ya da görsel belgesinin bağlantısını tutar. Metin içerdiği için varchar veri tipi kullanılmıştır.

- **expense_date:** Harcamaya ait tarih bilgisini saklar. Yalnızca tarih verisi içereceği için date veri tipi tercih edilmiştir.

- **created_at:** Harcamaların sisteme ne zaman eklendiğini belirtmek için timestamp veri tipi kullanılmıştır.

- **updated_at:** Harcama kaydının en son ne zaman güncellendiğini ifade eder. Bu alan da timestamp veri tipi ile tanımlanmıştır.

3.3.2.5. NotificationPreferences Tablosu

- **id (PK):** Her kullanıcının bildirim tercihlerini benzersiz şekilde tanımlamak amacıyla primary key olarak belirlenmiştir. Tekil kayıt yapısı sağlamak için UUID veri tipi kullanılmıştır.

- **user_id (FK):** Bildirim tercihlerinin hangi kullanıcıya ait olduğunu belirtmek için foreign key olarak tanımlanmıştır. Users tablosundaki id alanına referans eder. Veri tipi UUID'dir.

- **group_id (FK):** Bildirim tercihlerinin hangi gruba özel olduğunu belirtir. Groups tablosunun id alanına referans eder. Bu sayede kullanıcı her grup için farklı bildirim ayarlarına sahip olabilir. Veri tipi UUID olarak tanımlanmıştır.

- **expense_notifications_enabled:** Kullanıcının harcama bildirimlerini alıp almama tercihlerini belirler. İki değer (true/false) aldığı için boolean veri tipi kullanılmıştır.

- **expense_threshold:** Belirli bir tutarın üzerindeki harcamalarda bildirim gönderilmesini sağlamak için tanımlanmıştır. Sayısal bir değer olduğu için numeric veri tipi kullanılmıştır.

- **member_join_notifications_enabled:** Gruba yeni bir üye katıldığında bildirim gönderilip gönderilmeyeceğini belirler. Boolean yapıya uygun olarak boolean veri tipi seçilmiştir.

- **created_at:** Tercih kaydının oluşturulma zamanını tutmak için timestamp veri tipi kullanılmıştır.

- **updated_at:** Kullanıcı tercihleri güncellendiğinde kaydedilen zaman bilgisidir. Bu alan da timestamp veri tipi ile tanımlanmıştır.

3.3.2.6. Notifications Tablosu

- **id (PK):** Sistemde oluşturulan her bildirimin benzersiz bir şekilde tanımlanabilmesi için primary key olarak tanımlanmıştır. Sistem içinde göstergelerin ayrı ayrı izlenmesini sağlamak için UUID veri tipi kullanılmıştır.

- **user_id (FK):** Bildirimin hangi kullanıcıya gönderildiğini belirtir. Users tablosundaki id alanına referans eder. Veri tipi UUID'dir.

- **group_id (FK):** Bildirimin hangi grupta gerçekleşen bir olaya ait olduğunu belirtir. Groups tablosundaki id alanına referans eder. Veri tipi UUID olarak belirlenmiştir.

- **type:** Bildirimin türünü (harcama eklendi, üye katıldı vb.) belirtir. Metinsel ifade içerdiği için varchar(20) veri tipi kullanılmıştır.

- **title:** Bildirimin başlığını tutar. Kısa metin bilgisi içerdiğinden varchar veri tipi tercih edilmiştir.

- **message:** Bildirimin detaylı açıklamasını içeren metindir. Uzun metin saklamak için text veri tipi seçilmiştir.

- **data:** Bildirimle ilişkili ek bilgileri (JSON formatında) barındırır. Yapılandırılmış veriyi saklayabilmek için jsonb veri tipi kullanılmıştır.
- **is_read:** Bildirimin okunup okunmadığını belirtir. Boolean yapıda olduğu için boolean veri tipi ile tanımlanmıştır.

- **created_at:** Bildirimin oluşturulma zamanını saklar. Veri tipi timestamp'tır.

- **updated_at:** Bildirimin en son düzenlenme zamanını gösterir. Bu alan da timestamp veri tipi ile tanımlanmıştır.

3.3.2.7. UserGroupPreferences Tablosu

- **id (PK):** Kullanıcıların gruplara yönelik kişisel ayarlarının benzersiz şekilde saklanmasını sağlamak için primary key olarak tanımlanmıştır. UUID veri tipi seçilmiştir.

- **user_id (FK):** Bu tercihin hangi kullanıcıya ait olduğunu belirtir. Users tablosundaki id alanına foreign key olarak bağlanmıştır. UUID veri tipi kullanılmıştır.

- **group_id (FK):** Bu tercihin hangi gruba ait olduğunu belirtir. Groups tablosundaki id alanına referans eder. Veri tipi UUID'dir.

- **sort_order:** Kullanıcının grupları veya harcamaları hangi sırada görmek istediğini belirtir. Sıralama değeri sayısal olduğundan integer veri tipi kullanılmıştır.

- **custom_color:** Kullanıcının ilgili grup için özel bir renk belirlemesini sağlar. Metinsel renk kodları (örn. hex) kullanılacağı için varchar(10) veri tipi seçilmiştir.

- **created_at:** Kaydın oluşturulma zamanını tutmak için timestamp kullanılmıştır.

- **updated_at:** Kaydın en son ne zaman güncellendiğini gösterir. timestamp veri tipi ile tanımlanmıştır.

4. Veritabanı Sorguları

Bu bölümde projenin fiziksel tasarım aşamasında oluşturulan veri tabanı yapısı ve tablolar arasındaki ilişkiler, örnek SQL sorguları üzerinden ayrıntılı olarak açıklanmaktadır. Hazırlanan sorgular ile veri tabanı tasarımının doğru çalıştığı, tablolar arasındaki ilişkilerin sağlıklı bir biçimde kurulduğu ve sistemin temel işlevlerini yerine getirebildiği gösterilmektedir. Bu kapsamda iç içe sorgular, gruplama ve filtreleme işlemleri ve çoklu tablo ilişkilendirmeleri yapıları kullanılarak veri tabanının işlevselliği örneklendirilmiştir.

4.1 İç İçe (Nested) Sorgu Örneği

Sorgu, her bir grup için ortalama harcama tutarının üzerinde harcama yapan kullanıcıları tespit etmek amacıyla hazırlanmıştır. Alt sorgu ile grup bazlı ortalama harcama hesaplanmakta, ana sorgu ile ise bu ortalamanın üzerinde kalan harcamalar listelenmektedir.

Recent Expenses by User and Group

```sql
SELECT
  u.full_name,
  e.amount,
  g.name AS group_name
FROM expenses e
JOIN users u ON e.created_by = u.id
JOIN groups g ON e.group_id = g.id
WHERE e.amount > (
  SELECT AVG(amount)
  FROM expenses
  WHERE group_id = e.group_id
);
```

Results Chart Export Source Primary Database Role postgres Run CTRL

full_name	amount	group_name
Furkan Kızıloğlu	1000.00	Genel
Furkan Kızıloğlu	500.00	Genel
Furkan Kızıloğlu	20000.00	Ofisim
Furkan Kızıloğlu	699.00	Genel
Furkan Kızıloğlu	440.00	Genel
Kutluhan Özkan	5900.00	Kütüphane Ekibi

Ekran Görüntüsü 1: İç İçe Sorgu Örneği

Bu sorgu ile iç içe sorguların kullanımı gösterilmiş, aynı grup içerisindeki harcamalar arasında karşılaştırma yapılarak analiz edilebilir sonuçlar elde edilmiştir.

4.2 Group By ve Having İçeren Sorgu Örneği

Sorgu, gruplara ait toplam harcama tutarlarını hesaplamak ve belirli bir eşik değerin üzerinde harcamaya sahip grupları listelemek amacıyla hazırlanmıştır.

Recent Expenses by User and Group

```sql
SELECT
    g.name AS group_name,
    SUM(e.amount) AS total_expense
FROM expenses e
JOIN groups g ON e.group_id = g.id
GROUP BY g.name
HAVING SUM(e.amount) > 1000;
```

Results Chart Export Source Primary Database Role postgres Run CTRL

group_name	total_expense
Tatil Haziran	2500.00
Genel	3106.00
Ofisim	22265.00
Kütüphane Ekibi	6151.00
Tatil grubu	25000.00

Ekran Görüntüsü 2: Group By ve Having Sorgu Örneği

Group by ifadesi ile gruplara ait harcamalar gruplanmış, Having koşulu ile toplam harcaması belirlenen değerin üzerinde olan gruplar filtrelenmiştir. Bu sorgu, sistemin istatistiksel analiz yapabilme yeteneğini göstermektedir.

4.3 Join İşlemi İçeren Sorgu Örneği

Sorgu, kullanıcıların hangi gruplarda hangi harcamaları yaptığını listeleyerek tablolar arasındaki ilişkilerin doğruluğunu göstermektedir.

Recent Expenses by User and Group

```sql
SELECT
    u.full_name,
    g.name AS group_name,
    e.title AS expense_title,
    e.amount,
    e.expense_date
FROM expenses e
JOIN users u ON e.created_by = u.id
JOIN groups g ON e.group_id = g.id
ORDER BY e.expense_date DESC;
```

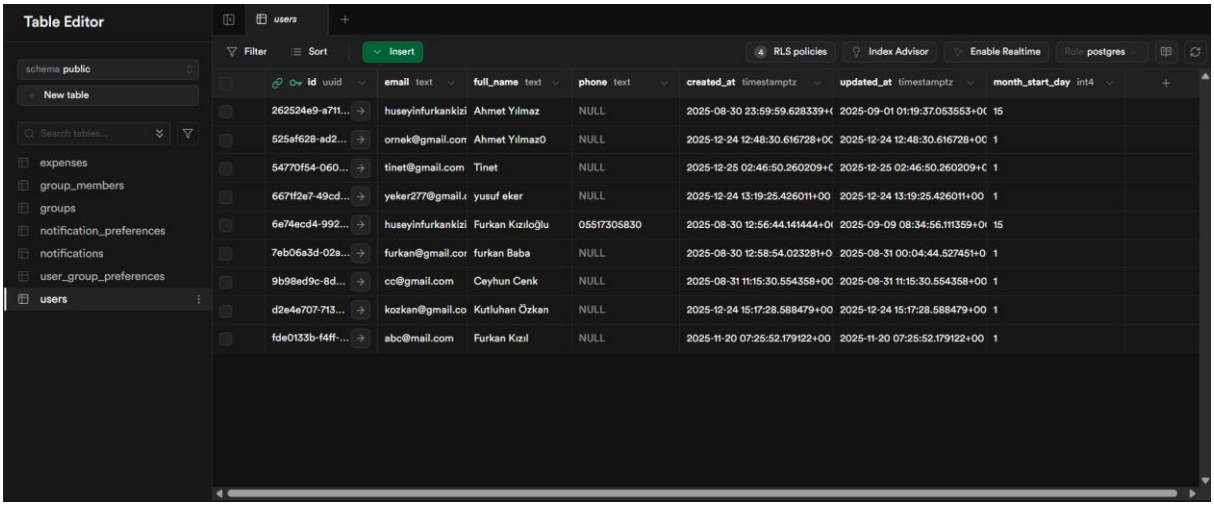
Results Chart Export Source Primary Database Role postgres Run CTRL

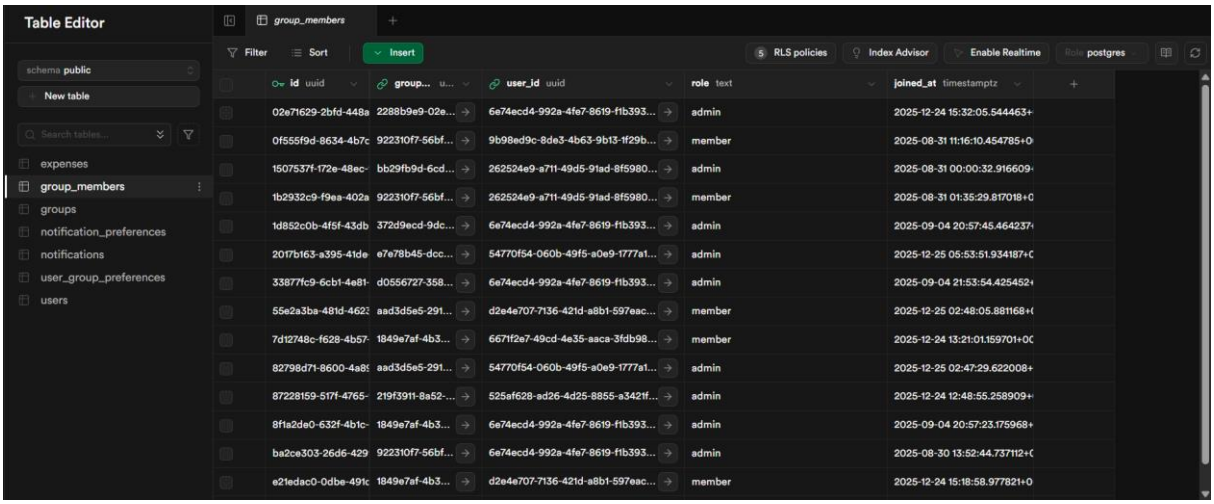
full_name	group_name	expense_title	amount	expense_date
Kutluhan Özkan	Kütüphane Ekibi	Rr	1.00	2025-12-25
Tinet	Spor Aletleri	Dumbell	900.00	2025-12-25
Kutluhan Özkan	Kütüphane Ekibi	Hokey Takımı	5900.00	2025-12-25
Kutluhan Özkan	Ofisim	Kaydırak	600.00	2025-12-25
Kutluhan Özkan	Ofisim	Oyuncak Araba	500.00	2025-12-25
Kutluhan Özkan	Ofisim	Yeni Mouse	650.00	2025-12-25

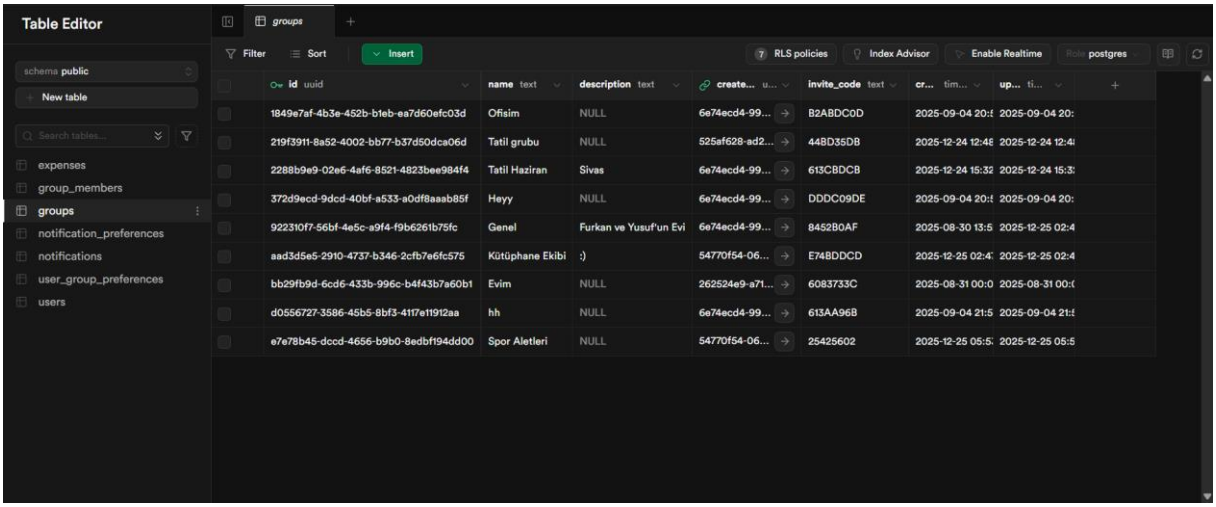
Ekran Görüntüsü 3: Join İşlemi İçeren Sorgu Örneği

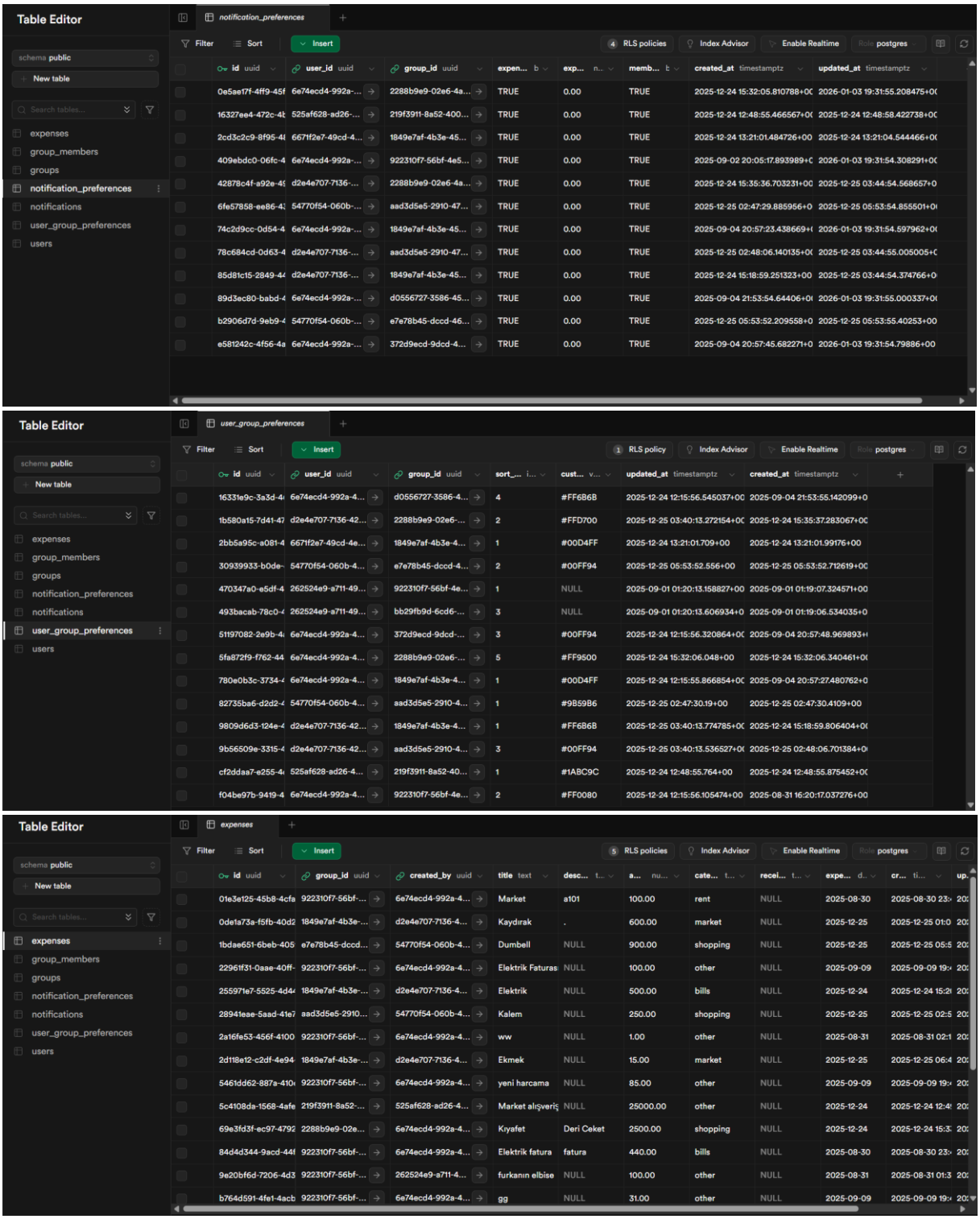
Bu sorgu ile expenses, users ve groups tabloları JOIN işlemleri kullanılarak ilişkilendirilmiş ve çoklu tablo yapısının doğru çalıştığı gösterilmiştir.

5. Veritabanı Ekran Görüntüleri



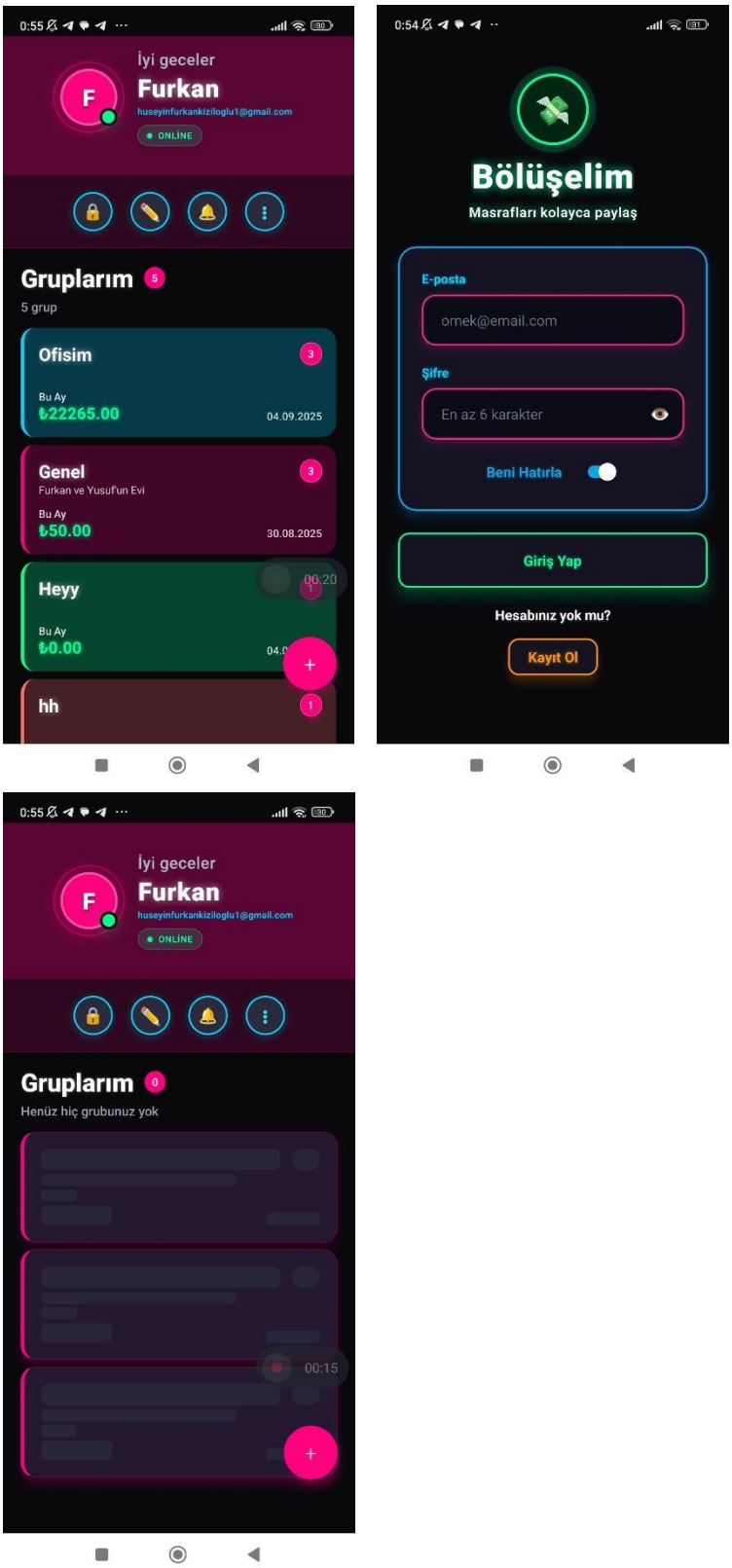






Ekran Görüntüsü 4: Veritabanı

6. Uygulama İçi Ekran Görüntüleri



Ekran Görüntüsü 5: Uygulama İçi Ekran Görüntüsü

7. Proje Yönetimi ve Görev Dağılımı

Bu projede ekip üyeleri arasında görev paylaşımı yapılarak proje yönetimi süreci planlı bir şekilde yürütülmüştür. Görev dağılımı aşağıda özetlenmiştir.

- **Yusuf Eker**: Projenin genel koordinasyonunu üstlenmiştir. Haftalık raporların ve genel raporların yazımı, proje dokümantasyonunun hazırlanması ve OneDrive ortamındaki dosyaların düzenli şekilde yönetilmesinden sorumlu olmuştur.
- **Emir Süha Baş:** Haftalık raporların hazırlanmasına katkı sağlamış ve projenin ön yüz (frontend) geliştirme sürecini yürütmüştür. Kullanıcı arayüzü tasarımı ve uygulamanın görsel bileşenlerinin oluşturulmasından sorumlu olmuştur.
- **Hüseyin Furkan Kızıloğlu:** Projenin arka uç (backend) geliştirme sürecini yürütmüş, veri tabanı ile uygulama arasındaki iş mantığının oluşturulmasına katkı sağlamıştır. Ayrıca proje fikrinin belirlenmesi ve proje seçimi aşamasında aktif rol almıştır.
- **Aykut Kocabıyık:** Proje süreci boyunca uygulama geliştirme ve dokümantasyon çalışmalarında aktif görev almamış, teknik geliştirme aşamalarına katkı sunmamıştır.

Görevler	Ağırlık	Yusuf Eker	Emir Süha Baş	Hüseyin Furkan Kızıloğlu	Aykut Kocabıyık	Durum (%)
Proje fikrinin belirlenmesi	2%	20,00%	30,00%	50,00%	0,00%	100,00%
Proje öneri raporunun hazırlanması	8%	50,00%	25,00%	25,00%	0,00%	100,00%
Haftalık raporların yazılması	8%	50,00%	30,00%	20,00%	0,00%	100,00%
Tasarım raporunun hazırlanması	8%	50,00%	25,00%	25,00%	0,00%	100,00%
Final raporunun hazırlanması	8%	50,00%	25,00%	25,00%	0,00%	100,00%
OneDrive / dosya yönetimi	8%	100,00%	0,00%	0,00%	0,00%	100,00%
Gereksinim analizi	3%	25,00%	50,00%	25,00%	0,00%	100,00%
Kavramsal veri tabanı tasarımı (ER Diyagramı)	5%	60,00%	0,00%	40,00%	0,00%	100,00%
Mantıksal veri tabanı tasarımı	3%	50,00%	0,00%	50,00%	0,00%	100,00%
Normalizasyon çalışmaları	3%	50,00%	0,00%	50%	0,00%	100,00%

Frontend geliştirme	15%	5,00%	90,00%	5,00%	0,00%	100,00%
Backend geliştirme	20%	5,00%	25,00%	70,00%	0,00%	100,00%
Veri tabanı tablolarının oluşturulması	5%	30,00%	30,00%	40,00%	0,00%	100,00%
SQL sorgularının yazılması ve test edilmesi	1%	0,00%	0,00%	100,00%	0,00%	100,00%
Test verilerinin oluşturulması	1%	0,00%	0,00%	100,00%	0,00%	100,00%
Logo Tasarımı	1%	0,00%	100,00%	0,00%	0,00%	100,00%
Ekran görüntülerinin alınması	1%	0,00%	0,00%	100,00%	0,00%	100,00%

TOPLAM	100,00%	34,40%	31,50%	34,10%	0,00%	100,00%

Contribuyion Matrix: SplitApp